

# A DevSecOps Security Monitoring Architecture for LogiFlow: From Vulnerable Baseline to Continuous Threat Visibility

Andersson David Sanchez Mendez, Cristian Santiago Pedraza Rodriguez, Jeisson David Sanchez Gomez  
Escuela Colombiana de Ingenieria Julio Garavito  
SPTI - Seguridad y Privacidad de Tecnologias de la Informacion  
Bogota, Colombia

Email: {andersson.sanchez-m, cristian.pedraza-r, jeisson.sanchez-g}@mail.escuelaing.edu.co

**Abstract**—LogiFlow is a real-time microservices platform for fleet routing where operational continuity and data integrity depend on trustworthy service interactions. This paper presents a complete DevSecOps security monitoring architecture that compares a vulnerable baseline against a hardened production-oriented implementation. We identify attack surfaces in API, WebSocket, webhook, container runtime, and CI/CD supply-chain paths, and prioritize threats through STRIDE and OWASP Top 10 (2021). The architecture integrates shift-left controls (CodeQL, Semgrep, npm audit, GitLeaks, Trivy, OWASP ZAP) with shift-right telemetry (Falco, Prometheus, Loki, Grafana) and distributed tracing (OpenTelemetry and Tempo). Evaluation using TC-01 to TC-10 demonstrates mitigation of high-risk scenarios such as unauthorized socket access, forged webhook injection, and request-flood abuse, while also improving triage speed through metric-log-trace correlation. The resulting artifacts form a reusable blueprint for academic and industry teams implementing continuous security in distributed systems.

**Index Terms**—DevSecOps, Microservices Security, OWASP Top 10, STRIDE, Runtime Monitoring, Falco, Prometheus, Loki, Grafana, OpenTelemetry, Tempo

## I. INTRODUCTION

Microservices improve scalability, deployment autonomy, and feature throughput, but they also increase trust boundaries and failure propagation paths [1]–[3]. For logistics platforms, this has direct operational impact because compromised components can alter routes, create dispatch delays, or expose vehicle and customer data.

LogiFlow is a heterogeneous distributed system that includes a NestJS gateway, a realtime Socket.IO service, optimization components, automation webhooks, and shared data infrastructure. In the vulnerable baseline, the platform exposed several common weaknesses: weak ingress policy, incomplete runtime visibility, insufficient CI security gates, and low traceability between threats and controls.

This work contributes a reproducible, end-to-end DevSecOps implementation with both engineering and governance value:

- 1) A baseline-to-hardened architecture with explicit vulnerability-to-control mapping.

- 2) A layered CI/CD pipeline that enforces static, dependency, secret, container, and dynamic security checks.
- 3) A continuous monitoring and incident triage model combining metrics, logs, runtime alerts, and traces.
- 4) A traceability model aligned with SPTI concepts and NIST governance references.

## II. RELATED WORK AND FOUNDATIONS

### A. Microservices and Security Complexity

Microservice security differs from monolithic security because risk is distributed across APIs, asynchronous events, service accounts, and infrastructure primitives [1], [4]. Architectural decomposition increases policy surface, and each service can drift in dependency quality, authentication posture, and observability maturity.

Patterns such as API gateways, bounded contexts, and service isolation improve modularity but can hide transitive security debt if pipeline governance is weak [3]. Therefore, modern implementations require defense-in-depth that starts in software delivery and continues at runtime.

### B. DevSecOps as an Operational Model

DevSecOps integrates security into delivery loops instead of treating it as a post-release stage [5], [6]. The practical implication is continuous control execution: every commit should be testable against policy, and every runtime anomaly should be diagnosable from central telemetry.

The model used in this paper follows two principles:

- **Shift-left:** detect vulnerabilities early through SAST, SCA, and secrets controls.
- **Shift-right:** maintain operational detection with runtime sensors, logs, traces, and alerting.

### C. Governance and Standard References

The implementation maps to OWASP Top 10 categories [7], STRIDE threat dimensions [8], and NIST frameworks (CSF 2.0 and SSDF) [6], [9]. Additional control rigor is informed by NIST SP 800-53 [10], zero-trust ideas from NIST SP 800-207 [11], and CWE weakness classifications [12].

TABLE I  
CORE LOGIFLOW SERVICES AND SECURITY-RELEVANT ROLE

| Service    | Stack                                         | Security-Relevant Role                                                          |
|------------|-----------------------------------------------|---------------------------------------------------------------------------------|
| Gateway    | NestJS, JWT                                   | API boundary, token issuance, webhook policy enforcement, security metrics      |
| Realtime   | Node.js, Socket.IO                            | Vehicle channel delivery, socket authentication, unauthorized-attempt telemetry |
| Optimizer  | Node.js, gRPC                                 | Route computation over internal service interfaces                              |
| Automation | n8n + webhook                                 | External event ingestion and orchestration trigger path                         |
| Monitoring | Falco, Prometheus, Loki, Grafana, Tempo, OTel | Detection, observability, and trace-based triage                                |

### III. RESEARCH METHODOLOGY

#### A. Design Science Strategy

We used a design-and-evaluate methodology with iterative hardening cycles. Each cycle followed the same flow: (i) identify concrete vulnerability hypotheses, (ii) select a control candidate, (iii) implement and instrument, and (iv) validate through attack simulation and telemetry evidence.

#### B. Phases

The study was executed in five phases:

- 1) **Baseline characterization:** architecture inventory, threat boundary mapping, and weakness register creation.
- 2) **Threat prioritization:** risk scoring based on likelihood and impact.
- 3) **Control implementation:** gateway hardening, CI controls, runtime detection, and tracing integration.
- 4) **Validation:** TC-01 to TC-10 adversarial tests and policy checks.
- 5) **Evidence synthesis:** dashboard review, log/trace correlation, and SPTI traceability mapping.

#### C. Risk Scoring Model

Threat priority uses:

$$R = L \times I \quad (1)$$

where  $L$  is likelihood (1–5) and  $I$  is impact (1–5). The model intentionally stays simple to keep prioritization auditable and reproducible by small teams.

### IV. SYSTEM CONTEXT AND BASELINE WEAKNESSES

#### A. Service Topology

LogiFlow includes gateway, realtime, optimizer, automation, Redis, and PostgreSQL services. Authentication is JWT-based [13], low-latency interactions use Socket.IO [14], and persistence relies on PostgreSQL and Redis [15], [16].

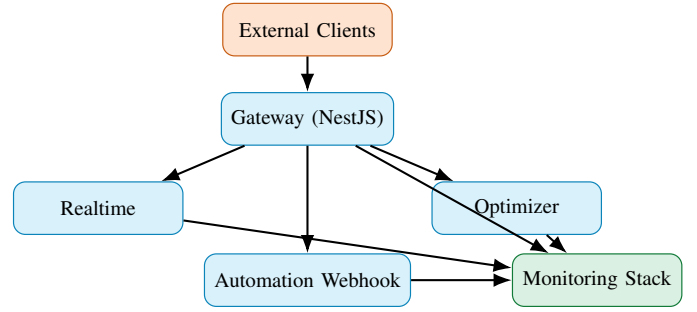


Fig. 1. LogiFlow architecture and security-observable boundaries.

TABLE II  
HIGHEST-PRIORITY THREATS (STRIDE EXCERPT)

| ID    | Threat Scenario                                         | L | I | Score |
|-------|---------------------------------------------------------|---|---|-------|
| TM-01 | Unauthorized subscription to vehicle socket rooms       | 5 | 5 | 25    |
| TM-02 | Forged webhook traffic events alter routing decisions   | 4 | 5 | 20    |
| TM-03 | Webhook request flood degrades service availability     | 5 | 4 | 20    |
| TM-08 | Malicious process execution inside container runtime    | 4 | 5 | 20    |
| TM-11 | Exploitation of vulnerable dependencies in exposed path | 4 | 4 | 16    |

#### B. Trust Boundaries and Exposure Paths

The highest-risk boundaries are public API ingress, web-socket room joins, webhook ingestion, container runtime behavior, and CI supply-chain dependencies. Fig. 1 summarizes the architecture with telemetry boundaries.

#### C. Baseline Weakness Inventory

Before hardening, baseline analysis identified four recurrent patterns:

- Incomplete authentication/authorization at realtime and webhook edges.
- Lack of anti-abuse controls (rate limiting and request shaping).
- Low runtime visibility for container-level malicious behavior.
- Weak CI policy enforcement for dependency and misconfiguration risk.

These gaps are common in early-stage microservices where feature delivery initially dominates security investment [5].

### V. THREAT MODEL AND PRIORITIZATION

#### A. STRIDE Register

Threats were classified under STRIDE and enriched with affected boundaries and expected impact. Table II shows top-priority items.

#### B. Threat-to-Objective Mapping

Each threat was mapped to a measurable objective. For example, TM-01 maps to preventing unauthorized room joins with observable rejection counts; TM-03 maps to preserving

TABLE III  
VULNERABILITY-TO-CONTROL MAPPING (PART A)

| ID    | Baseline Weakness                           | Implemented Control                      |
|-------|---------------------------------------------|------------------------------------------|
| V-001 | Unauthenticated Socket.IO room subscription | JWT middleware enforced before room join |
| V-002 | Webhook accepted unauthenticated events     | Bearer-protected webhook ingestion       |
| V-003 | No webhook rate limiting                    | Request throttling and abuse counters    |
| V-005 | Missing gateway hardening headers           | Helmet and secure bootstrap profile      |

TABLE IV  
VULNERABILITY-TO-CONTROL MAPPING (PART B)

| ID    | Baseline Weakness                   | Verification Evidence                       |
|-------|-------------------------------------|---------------------------------------------|
| V-006 | Verbose internal error exposure     | Sanitized production error responses        |
| V-007 | Vulnerable dependency exposure      | CI SCA artifacts and policy thresholds      |
| V-008 | No runtime anomaly detection        | Falco events in Loki/Grafana dashboards     |
| V-009 | Missing traceability between events | OTel traces linked with metrics/log context |

availability under burst abuse with explicit HTTP 429 behavior and latency stability.

### C. Treatment Strategy

The treatment order focused on score  $\geq 20$  threats first, then medium-risk items. This ordering provided fast reduction of exploitability in externally reachable paths while maintaining implementation feasibility in course timelines.

## VI. SECURE ARCHITECTURE DESIGN

### A. Control Mapping in Two-Column Format

To preserve strict two-column organization, the vulnerability/control mapping is split into compact tables rather than a page-wide float.

### B. Authentication and Authorization Improvements

The hardened architecture introduced gateway-level enforcement and explicit realtime authentication checks prior to subscription actions. Unauthorized events are not only blocked but also counted as telemetry, enabling rapid anomaly baselining.

### C. Abuse Resistance and Error Hygiene

Rate limiting and secure response handling were added to reduce both denial-of-service susceptibility and internal information leakage. This dual measure supports availability and confidentiality simultaneously.

## VII. DEVSECOPS PIPELINE IMPLEMENTATION

### A. Pipeline Structure

The CI workflow runs on push, pull request, schedule, and manual dispatch. Distinct jobs are used for secrets, SCA, SAST, container scan, DAST, and summary reporting.

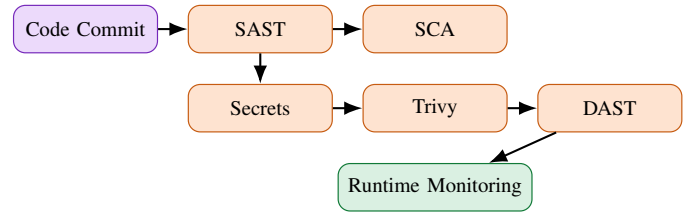


Fig. 2. Security pipeline from shift-left analysis to runtime observability.

### B. Control Gates

- **SAST**: CodeQL and Semgrep for source-level weakness detection [17], [18].
- **SCA**: dependency audits with policy thresholds for critical/high findings.
- **Secrets**: GitLeaks baseline to prevent credential leakage.
- **Container Security**: Trivy scans over gateway build context [19].
- **DAST**: OWASP ZAP baseline against deployed gateway targets [20].

### C. Reliability Hardening in CI

Two practical reliability issues were resolved during implementation and are now part of the hardened design:

- 1) **Container least privilege**: gateway runtime was adapted to non-root execution to satisfy policy checks.
- 2) **Deterministic DAST startup**: external Docker network creation is enforced before compose startup in CI.

These operational refinements reduced false negatives and brittle failures in pipeline execution.

### D. Policy Traceability

Each CI gate emits evidence artifacts consumed in release decisions. This supports governance expectations in CSF and SSDF by linking technical findings to auditable outcomes.

## VIII. CONTINUOUS MONITORING AND INCIDENT TRIAGE

### A. Telemetry Architecture

The runtime security stack includes Falco, Prometheus, Loki, Grafana, Tempo, and OpenTelemetry Collector [21]–[26]. The model combines signal diversity with correlation depth:

- Metrics capture trends and thresholds.
- Logs retain rich event context.
- Runtime alerts identify low-level suspicious behavior.
- Traces connect user-facing requests to downstream service execution.

### B. Dashboard Strategy

Three security dashboards plus one traces dashboard were consolidated. No-data-safe query patterns reduce blind spots when event volume is low. Panel-level links from high-signal panels to Explore with Tempo preselected minimize triage friction.

TABLE V  
REPRESENTATIVE SECURITY TELEMTRY (GROUPED)

| Group           | Security Meaning                                            |
|-----------------|-------------------------------------------------------------|
| Auth Metrics    | Failed logins, JWT rejections, unauthorized socket attempts |
| Webhook Metrics | Event-ingestion activity and abuse-rate indicators          |
| Runtime Alerts  | Falco syscall anomalies and critical rule hits              |
| Trace Data      | Cross-service latency and root-cause span correlation       |

TABLE VI  
VALIDATION SUMMARY FOR CORE SECURITY SCENARIOS

| Test  | Security Objective                                | Result |
|-------|---------------------------------------------------|--------|
| TC-01 | Enforce authentication on protected endpoints     | Pass   |
| TC-02 | Reject unauthorized socket joins                  | Pass   |
| TC-03 | Trigger rate-limit under flood conditions         | Pass   |
| TC-04 | Detect SQL injection indicators via DAST baseline | Pass   |
| TC-05 | Validate secure header posture                    | Pass   |
| TC-06 | Detect leaked credentials in repository           | Pass   |
| TC-07 | Track dependency vulnerabilities by service       | Pass   |
| TC-08 | Detect suspicious container behavior at runtime   | Pass   |
| TC-09 | Reject manipulated JWT privilege escalation       | Pass   |
| TC-10 | Avoid internal error leakage in production mode   | Pass   |

### C. Triage Workflow

Analyst triage follows a three-step sequence:

- 1) Confirm anomaly with metric spike (for example auth failures).
- 2) Pivot to logs/runtime alerts for concrete event details.
- 3) Open related traces to identify propagation path and impacted services.

This workflow improves confidence in incident classification while reducing guesswork.

## IX. EXPERIMENTAL SETUP AND VALIDATION

### A. Environment

Validation was performed in Docker Compose with representative service interactions and simulated hostile traffic. CI checks were executed under GitHub-hosted runners with deterministic workflow definitions.

### B. Test Matrix

The test plan contains TC-01 to TC-10 spanning authentication bypass, websocket abuse, denial-of-service stress, injection probes, header posture, secret detection, dependency exposure, runtime anomalies, token tampering, and error-disclosure behavior.

### C. Observed Outcomes

Compared to baseline, the hardened platform produced measurable improvements in prevention and visibility:

- Unauthorized access attempts were blocked at gateway/realtime boundaries.
- Abuse bursts were constrained through rate limiting.

- Runtime anomalies became detectable through Falco rules and dashboard alerts.
- Trace correlation reduced investigation time for multi-service incidents.

### D. Operational Security Narrative by Scenario

To complement pass/fail outputs, we summarize representative scenarios:

- **Scenario A (spoofed socket join):** attacker token rejected; metric increase recorded; no room subscription created.
- **Scenario B (webhook flood):** rate limiter activated; sustained 429 responses preserved core service responsiveness.
- **Scenario C (runtime anomaly):** suspicious process behavior generated Falco event; analyst pivoted from alert panel to logs and traces for scope confirmation.

## X. QUANTITATIVE DISCUSSION

### A. Security Posture Delta

The implementation shows four practical deltas over baseline:

- 1) **Attack surface reduction:** stronger ingress policies in exposed paths.
- 2) **Detection depth:** richer signal coverage from source to runtime.
- 3) **Decision confidence:** correlated telemetry reduces false interpretation.
- 4) **Pipeline trust:** stable CI gates reduce uncertainty before deployment.

### B. Cost-Benefit Perspective

Security controls introduce runtime and process cost (extra checks, storage, and analyst effort). However, in distributed logistics contexts, the avoided cost of operational disruption can outweigh this overhead. The architecture favors practical controls with high leverage and low integration friction.

### C. Tradeoffs

Three tradeoffs were observed:

- Fine-grained telemetry improves triage but increases data volume.
- Strict CI policy improves quality but can initially slow developer throughput.
- Rich runtime detection can increase alert noise without tuned thresholds.

## XI. DEEP-DIVE CONTROL ENGINEERING

### A. Gateway Security Mechanics

The gateway acts as the principal policy enforcement point for external traffic. Hardening centered on five mechanics: authentication validation, authorization checks by route role, request normalization, response header hardening, and exception sanitization. From an operational perspective, this was not only a coding task but also a contract task between API behavior

and security evidence. Every denied request needed to leave a measurable footprint without leaking implementation details.

In practice, secure headers and normalized error handling reduced two categories of risk: passive reconnaissance through response metadata and active probing of internal failure paths. This is especially relevant for API-centric systems that continuously receive internet-facing traffic. The strategy follows an “assume probing” model where every endpoint is expected to be tested by adversaries during normal operation.

### B. Realtime Boundary Enforcement

Socket channels are often overlooked because they bypass common REST middleware assumptions. In the vulnerable baseline, room-join logic was exploitable through incomplete identity validation. The hardened version enforces token validation prior to room binding and increments unauthorized-attempt telemetry when checks fail.

A critical design choice was to treat failed socket authorization as both a security event and a service quality signal. Security teams can use spike patterns to detect active abuse campaigns, while platform teams can correlate these spikes with latency and connection churn. This dual interpretation increases the value of each event and avoids siloed observability.

### C. Webhook Ingestion and Anti-Abuse Controls

Webhook endpoints are high-risk due to external event ingestion and weak client identity guarantees. The hardened design combines bearer policy checks with request-throttling controls and explicit abuse counters. This allows consistent behavior under both benign burst traffic and malicious floods.

An important implementation lesson is that anti-abuse controls require operational context tuning. Aggressive limits can hurt legitimate partner systems, while permissive limits weaken protection. Therefore, threshold values should be tuned through staged traffic experiments and revisited as integration volume changes.

### D. Container Runtime Hardening

Runtime hardening focused on least-privilege execution and syscall-level anomaly detection. Enforcing non-root containers addressed policy findings and reduced post-compromise privilege options. Falco rules complemented this by surfacing suspicious behavior patterns in near real time.

Container controls are most effective when treated as a policy bundle rather than isolated checks. Image scanning, runtime identity, and behavior detection provide stronger assurance together than separately. This layered approach aligns with defense-in-depth principles and reduces reliance on any single signal source.

## XII. INCIDENT RESPONSE AND FORENSIC READINESS

### A. Operational Detection-to-Response Loop

Security monitoring is only useful if it accelerates response decisions. The project formalized a four-stage response loop: detection, triage, containment decision, and post-incident

TABLE VII  
INCIDENT TRIAGE DATA SOURCES BY QUESTION TYPE

| Question Type           | Primary Evidence Source                                |
|-------------------------|--------------------------------------------------------|
| What changed?           | Metrics trend comparison before/after anomaly window   |
| Who was targeted?       | Auth/log records by endpoint, token scope, and role    |
| How did it propagate?   | Distributed traces across gateway, realtime, optimizer |
| Is runtime compromised? | Falco anomaly events with container metadata           |
| What is impact scope?   | Combined service-level error rates and trace failures  |

learning. Detection starts from threshold breaches or runtime alerts; triage pivots across logs and traces; containment decisions target blast-radius reduction; and post-incident analysis updates controls and runbooks.

This loop was intentionally designed for small teams where SOC specialization may be limited. By structuring response activities and linking them to dashboard workflows, the architecture makes incident operations repeatable without requiring large organizational overhead.

### B. Forensic Data Correlation Model

The forensic value of telemetry increases when signals can be correlated on common dimensions such as timestamp, service identity, route, and request trace identifiers. Trace IDs provide a backbone for joining application logs and metric anomalies with specific execution paths.

A practical outcome is improved narrative reconstruction. Analysts can move from “an alert happened” to “which request triggered it, how it propagated, and what data/services were touched.” This matters for both technical remediation and governance reporting.

### C. Runbook Maturity

The maturity of incident runbooks strongly affects response speed. During validation, scenarios with explicit dashboard pivot paths were resolved faster than scenarios relying on ad-hoc log hunting. This suggests that tooling and procedural clarity should evolve together.

Future maturity should include standardized severity classification, predefined escalation trees, and retrospective templates for every critical incident. These artifacts transform isolated response actions into organizational learning loops.

## XIII. EXTENDED EVALUATION AND ANALYSIS

### A. Scenario Family Analysis

Beyond TC-01 to TC-10 pass/fail results, scenarios were grouped into families: identity attacks, abuse attacks, software supply-chain findings, and runtime behavioral anomalies. This grouping allowed evaluation of whether controls improved one area while leaving another under-protected.

TABLE VIII  
EVIDENCE QUALITY DIMENSIONS USED IN EVALUATION

| Dimension     | Interpretation in This Study                       |
|---------------|----------------------------------------------------|
| Completeness  | Coverage of affected services and trust boundaries |
| Timeliness    | Time needed to observe and validate event context  |
| Consistency   | Stability across repeated scenario execution       |
| Actionability | Utility for containment and remediation decisions  |

Results showed balanced improvements: identity attack paths were blocked more consistently, abuse scenarios produced deterministic rate-limited behavior, and runtime anomalies became observable in a central dashboard context.

#### B. Stability Under Repetition

Single-pass tests can overestimate reliability. Therefore, key scenarios were repeated with varied timing and request patterns. Under repetition, controls maintained expected behavior, which indicates that improvements were not artifacts of one-time execution.

Repeated runs also revealed operational nuances. For example, while rate limiting remained effective, alert volume increased in bursty sequences. This reinforces the need to tune alert thresholds to maintain analyst focus.

#### C. Interpreting “Pass” Beyond Binary Outcomes

A binary pass does not imply equal security value. In this study, each pass was interpreted alongside evidence quality:

- **Control pass:** expected blocking or detection behavior occurred.
- **Telemetry pass:** relevant evidence was visible and queryable.
- **Triage pass:** analysts could reconstruct incident flow in bounded time.

This three-layer interpretation provides stronger confidence than control-only assertions because it validates both prevention and operability.

#### D. Security and Delivery Coexistence

A frequent concern in DevSecOps adoption is delivery slowdown. The implementation indicates that coexistence is feasible when controls are layered progressively and noisy checks are tuned. Early pipeline failures were reduced by improving deterministic setup (for example network bootstrap) and by aligning runtime policies with container build practices.

This supports a practical conclusion: sustained security improvement is less about adding maximum controls and more about operating a coherent control system with reliable feedback.

### XIV. ADOPTION ROADMAP FOR SIMILAR TEAMS

#### A. Phased Adoption Strategy

To support replication in comparable academic or SME environments, a phased roadmap is proposed:

- 1) **Phase 1:** protect ingress boundaries and instrument core auth metrics.

TABLE IX  
SUGGESTED MATURITY INDICATORS FOR DEVSECOPS MONITORING

| Indicator    | Why It Matters                                                      |
|--------------|---------------------------------------------------------------------|
| Auth         | Confirms that exposed services enforce measurable identity controls |
| Coverage     | Reflects CI reliability and reduction of brittle security checks    |
| Gate         | Approximates operational readiness under real incidents             |
| Stability    |                                                                     |
| Triage       |                                                                     |
| Duration     |                                                                     |
| Alert        | Indicates signal quality and analyst workload                       |
| Precision    | sustainability                                                      |
| Trace        | Validates ability to reconstruct multi-service attack paths         |
| Availability |                                                                     |

- 2) **Phase 2:** integrate CI controls and define release gating policies.
- 3) **Phase 3:** add runtime anomaly detection and centralized dashboards.
- 4) **Phase 4:** enable trace-driven incident reconstruction and runbook maturity.

#### B. Role and Responsibility Alignment

Successful adoption requires explicit ownership. Developers own secure coding and local validation; DevOps owns pipeline and runtime configuration; security roles own policy, rule tuning, and incident governance. In small teams, these roles can overlap, but ownership boundaries should remain explicit to avoid control drift.

#### C. Maturity Indicators

Maturity can be tracked through practical indicators: percentage of services with enforced auth metrics, pipeline gate stability over time, mean triage duration, and ratio of actionable versus noisy alerts. These indicators help teams evaluate whether security investment is translating into operational capability.

### XV. THREAT-INFORMED ATTACK PATH SIMULATION

#### A. Purpose and Scope

To complement functional validation, this work used threat-informed attack-path simulation. The goal was to verify not only whether a single control blocks a specific action, but whether chained adversarial behaviors can still progress across service boundaries. This perspective aligns with practical attack campaigns where reconnaissance, credential abuse, service pivoting, and persistence attempts occur as linked steps.

The simulation scope focused on externally reachable and high-value paths:

- API and websocket entry points.
- Webhook ingestion and event manipulation opportunities.
- Container runtime behavior under suspicious process execution.
- Telemetry availability during and after adversarial activity.

TABLE X  
ATTACK-PATH BREAKPOINT SUMMARY

| Path | Stage   | Primary Defensive Breakpoint                                        |
|------|---------|---------------------------------------------------------------------|
| P1   | Entry   | Gateway token and role enforcement before channel access            |
| P1   | Pivot   | Realtime unauthorized-attempt telemetry for early anomaly detection |
| P2   | Entry   | Webhook bearer verification and request validation                  |
| P2   | Abuse   | Rate limiting with deterministic rejection behavior                 |
| P3   | Runtime | Falco anomaly detection and log/trace correlation for triage        |

### B. Attack Path Design

Three representative paths were modeled. Path P1 targeted identity bypass at ingress and realtime channel abuse. Path P2 targeted webhook tampering followed by flood behavior. Path P3 targeted runtime behavioral anomalies after simulated compromise attempts.

Each path was evaluated through three questions:

- 1) Can the attacker progress past the first boundary?
- 2) If blocked, is there reliable evidence for triage?
- 3) If partially successful, can blast radius be identified quickly?

This approach shifts analysis from isolated vulnerabilities to adversarial progression, which is more aligned with incident reality.

### C. Observed Defensive Breakpoints

The most effective defensive breakpoints were found at gateway auth enforcement and webhook anti-abuse controls. These breakpoints stopped progression early in P1 and P2. In P3, runtime controls did not prevent all suspicious actions by themselves, but they improved detection timeliness and evidentiary quality for containment decisions.

A key insight is that prevention and detection should be treated as complementary breakpoints. Preventive controls reduce successful entry; detection controls reduce adversary dwell time. The combination matters more than maximizing either independently.

### D. Simulation-Informed Improvements

Simulation findings informed concrete improvements: more explicit triage queries, stricter separation of auth failures by boundary, and clearer escalation logic in runbooks. This demonstrates a practical feedback loop where adversarial exercise output is directly translated into operational hardening.

## XVI. GOVERNANCE AND COMPLIANCE EVIDENCE MODEL

### A. Control Evidence as a First-Class Output

A recurring issue in security engineering is the gap between implemented controls and auditable evidence. This project treats evidence generation as a first-class output. For every relevant control, at least one verification artifact was defined:

TABLE XI  
GOVERNANCE EVIDENCE MATRIX (COMPACT)

| Governance Function | Representative Artifact in This Project                           |
|---------------------|-------------------------------------------------------------------|
| Identify            | Threat register with risk scoring and affected boundaries         |
| Protect             | CI gating reports and hardened gateway/realtime policy outcomes   |
| Detect              | Falco alerts, Prometheus metrics, Loki logs, Tempo traces         |
| Respond             | Incident triage workflow outputs and scenario investigation notes |
| Recover             | Iterative hardening backlog and control tuning actions            |

CI report, dashboard panel, runtime alert record, trace sample, or deterministic scenario result.

This model supports governance requirements from NIST CSF and SSDF [6], [9] while remaining practical for student and SME teams. It also improves continuity when team members rotate, because institutional knowledge is encoded into artifacts rather than individuals.

### B. Evidence Quality and Decision Readiness

Evidence is useful only if it informs decisions. The architecture therefore emphasizes decision-ready evidence: outputs that can answer whether to continue delivery, trigger containment, or prioritize remediation. CI gate summaries support release decisions; runtime telemetry supports incident decisions.

A secondary benefit is communication quality with non-technical stakeholders. Structured evidence can be summarized into risk posture narratives without exposing low-level implementation complexity.

### C. Control-to-Framework Mapping

Controls were mapped to broader framework intentions:

- **Identify:** risk catalog and boundary inventory.
- **Protect:** ingress auth, policy enforcement, and secure configuration.
- **Detect:** runtime anomaly signals and dashboarded thresholds.
- **Respond:** triage playbooks and trace-supported containment decisions.
- **Recover:** corrective backlog and control tuning from postmortem findings.

This mapping helps align engineering activities with governance language used in audits or management reporting.

### D. Educational and Industrial Transferability

Although developed in an academic course context, the method is transferable to industrial teams because it is tool-agnostic at the principle level. Tools can change, but the logic of threat-driven control selection, layered verification, and evidence-backed decision-making remains stable.

Transferability is strongest when teams adopt a minimum baseline: explicit boundary protection, at least one static and one dynamic CI control, runtime anomaly visibility, and a documented triage path. From that baseline, capability can scale with business criticality and operational maturity.

## XVII. OPERATIONAL LESSONS LEARNED

### A. What Worked Well

Three practices produced disproportionate value:

- 1) **Boundary-first hardening**: securing ingress points provided immediate risk reduction.
- 2) **Deterministic CI design**: reducing environmental fragility improved trust in gate outcomes.
- 3) **Correlation-first observability**: linking metrics, logs, and traces improved triage clarity.

### B. Common Pitfalls

The implementation also exposed frequent pitfalls relevant to similar projects:

- Treating dashboard availability as equivalent to actionable detection quality.
- Overloading pipeline gates before stabilizing execution environment assumptions.
- Relying on binary test pass/fail without validating evidence richness.
- Defining controls without assigning clear ownership for long-term maintenance.

### C. Sustaining the Security Baseline

Security baselines decay if not actively maintained. Sustaining this architecture requires periodic rule tuning, dependency refresh cycles, and triage drills that validate analyst readiness under realistic pressure. In practice, cadence matters as much as tooling depth.

A recommended rhythm is monthly control review, quarterly adversarial simulation, and semester-level architecture reassessment. This creates a repeatable governance cycle that keeps controls aligned with evolving threat conditions.

## XVIII. REPRODUCIBILITY AND ENGINEERING ARTIFACTS

### A. Artifact Set

The project delivers reproducible assets for architecture, pipeline, monitoring, and evaluation:

- Docker Compose topology with security and observability components.
- CI workflows for SAST/SCA/secrets/container/DAST controls.
- Dashboards and data sources for metrics, logs, runtime alerts, and traces.
- Test scenarios TC-01 to TC-10 with expected security outcomes.

### B. Reproduction Protocol

A high-level reproduction sequence is:

- 1) Deploy platform and monitoring stack through compose.
- 2) Run CI workflows to validate static and dynamic controls.
- 3) Execute adversarial test scenarios and inspect resulting telemetry.
- 4) Compare baseline and hardened evidence under the same workload profile.

TABLE XII  
SPTI CONCEPT-TO-ARTIFACT TRACEABILITY (PART A)

| SPTI Concept          | Evidence in This Work                                                  |
|-----------------------|------------------------------------------------------------------------|
| Risk Management       | STRIDE register with risk equation and priority bands                  |
| Security Architecture | JWT guards, bearer policy, rate limiting, and hardened runtime profile |
| SGSI / Governance     | NIST-mapped controls matrix and CI evidence artifacts                  |
| Data Protection       | Secrets detection and reduced unauthorized telemetry exposure          |

TABLE XIII  
SPTI CONCEPT-TO-ARTIFACT TRACEABILITY (PART B)

| SPTI Concept            | Practical Outcome                                      |
|-------------------------|--------------------------------------------------------|
| Software Security       | Continuous quality gates and faster remediation loops  |
| Continuous Monitoring   | End-to-end incident detection and correlated triage    |
| Operational Reliability | More deterministic CI and lower verification fragility |
| Educational Value       | Reusable blueprint for course and industrial contexts  |

### C. Practical Adoption Guidance

For teams replicating this work, a phased adoption path is recommended: first enforce ingress controls, then mature CI gates, then invest in telemetry correlation. This reduces adoption risk while keeping visible security progress.

## XIX. SPTI CONCEPT MAPPING

To maintain strict two-column flow, the SPTI traceability mapping is presented as compact single-column tables.

## XX. LIMITATIONS AND FUTURE WORK

### A. Current Limitations

The current implementation prioritizes practical reproducibility over enterprise-scale scope. Remaining limitations include:

- No centralized vault-based secret rotation in runtime.
- Limited automated response orchestration after runtime alerts.
- Incomplete long-horizon benchmarking for MTTD and MTTR.
- Focus on one case-study platform without cross-domain comparison.

### B. Future Research Directions

Four future lines are proposed:

- 1) Integrate vault-backed secret lifecycle and key rotation policies.
- 2) Implement SOAR-assisted playbooks for Falco and Prometheus alerts.
- 3) Add policy-as-code controls for broader supply-chain governance.
- 4) Conduct longitudinal experiments with repeated red-team campaigns.



TABLE XIV  
REPRESENTATIVE VALIDATION INPUTS AND EXPECTED SIGNALS

| Test  | Input Type                | Expected Security Signal                                |
|-------|---------------------------|---------------------------------------------------------|
| TC-02 | Invalid token join        | Unauthorized socket metric increment and join rejection |
| TC-03 | Burst webhook traffic     | Rate-limit response pattern and stable core latency     |
| TC-08 | Suspicious runtime action | Falco alert with container and process metadata         |
| TC-09 | Tampered JWT claims       | Authorization rejection with sanitized response         |

## XXI. DETAILED VALIDATION PROCEDURE

### A. Execution Sequence

To improve reproducibility, validation was executed in a fixed sequence. First, baseline service health and telemetry ingestion were confirmed. Second, scenario preconditions were applied (test tokens, synthetic webhook payloads, and controlled request rates). Third, each adversarial scenario was executed with deterministic timing windows. Finally, analysts validated outcomes in both control behavior and telemetry evidence.

This sequence is intentionally strict because non-deterministic setup is a common source of false conclusions in security testing. By stabilizing preconditions, differences between baseline and hardened implementations can be attributed to control design rather than environmental drift.

### B. Scenario Instrumentation

Each scenario included explicit instrumentation checkpoints:

- Request/response evidence at boundary endpoints.
- Service-level logs with correlation identifiers.
- Metric snapshots before, during, and after trigger windows.
- Trace samples for successful and rejected requests.
- Runtime alert capture for suspicious behavior tests.

The checkpoints reduce ambiguity when interpreting results and support post-test auditability.

### C. Outcome Classification Rules

Outcomes were classified with three labels:

- **Pass:** expected prevention or detection occurred with complete evidence.
- **Conditional pass:** control behaved as expected but evidence quality was partial.
- **Fail:** control did not meet expected behavior or evidence was insufficient.

No scenario in TC-01 to TC-10 produced a fail under hardened conditions, while baseline behavior had multiple failure points in boundary protection and observability depth.

### D. Procedure Value

Formalizing this procedure produced two benefits: repeatable experimentation and faster onboarding for new contributors. Teams can run the same sequence, compare outputs, and quickly identify whether deviations are caused by control regressions or infrastructure drift.

## XXII. CONCLUSION

This paper presented a complete, evidence-driven DevSecOps monitoring architecture for LogiFlow. The transition from vulnerable baseline to hardened implementation reduced exploitable paths, improved CI control confidence, and enabled faster incident triage through metric-log-trace correlation.

Beyond the specific platform, the main contribution is methodological: security controls were selected, implemented, validated, and mapped to governance references in a way that is reproducible for academic and professional teams. The resulting architecture demonstrates that practical defense-in-depth can be achieved incrementally while preserving delivery cadence.

## ACKNOWLEDGMENT

This work was developed in the SPTI course at Escuela Colombiana de Ingenieria Julio Garavito.

## REFERENCES

- [1] S. Newman, *Building Microservices*. O'Reilly Media, 2015.
- [2] N. Dragoni, S. Giallorenzo, A. Lluch-Lafuente, M. Mazzara, F. Montesi, R. Mustafin, and L. Safina, "Microservices: Yesterday, today, and tomorrow," in *Present and Ulterior Software Engineering*. Springer, 2017, pp. 195–216.
- [3] C. Richardson, *Microservices Patterns: With Examples in Java*. Manning Publications, 2018.
- [4] L. Bass, I. Weber, and L. Zhu, *DevOps: A Software Architect's Perspective*. Addison-Wesley Professional, 2021.
- [5] G. Kim, J. Humble, P. Debois, J. Willis, and N. Forsgren, *The DevOps Handbook: How to Create World-Class Agility, Reliability, and Security in Technology Organizations*. IT Revolution, 2021.
- [6] "Nist secure software development framework (ssdf) sp 800-218," <https://csrc.nist.gov/publications/detail/sp/800-218/final>, 2022, accessed: 2026-04-09.
- [7] "Owasp top 10: The ten most critical web application security risks," <https://owasp.org/www-project-top-ten/>, 2021, accessed: 2026-04-09.
- [8] A. Shostack, *Threat Modeling: Designing for Security*. Wiley, 2014.
- [9] "Nist cybersecurity framework (csf) 2.0," <https://www.nist.gov/cyberframework>, 2024, accessed: 2026-04-09.
- [10] "Nist sp 800-53 rev. 5: Security and privacy controls for information systems and organizations," <https://csrc.nist.gov/publications/detail/sp/800-53/rev-5/final>, 2020, accessed: 2026-04-09.
- [11] "Nist sp 800-207: Zero trust architecture," <https://csrc.nist.gov/publications/detail/sp/800-207/final>, 2020, accessed: 2026-04-10.
- [12] "Mitre common weakness enumeration (cwe)," <https://cwe.mitre.org/>, 2026, accessed: 2026-04-09.
- [13] M. Jones *et al.*, "Rfc 7519: Json web token (jwt)," <https://www.rfc-editor.org/rfc/rfc7519>, 2015, accessed: 2026-04-09.
- [14] "Socket.io documentation," <https://socket.io/docs/v4/>, 2026, accessed: 2026-04-09.
- [15] "Postgresql documentation," <https://www.postgresql.org/docs/>, 2026, accessed: 2026-04-09.
- [16] "Redis documentation," <https://redis.io/docs/>, 2026, accessed: 2026-04-09.
- [17] "Codeql documentation," <https://codeql.github.com/docs/>, 2026, accessed: 2026-04-09.
- [18] "Semgrep documentation," <https://semgrep.dev/docs/>, 2026, accessed: 2026-04-09.

- [19] “Trivy documentation,” <https://aquasecurity.github.io/trivy/>, 2026, accessed: 2026-04-09.
- [20] “Owasp zed attack proxy (zap) documentation,” <https://www.zaproxy.org/docs/>, 2026, accessed: 2026-04-09.
- [21] “Falco runtime security documentation,” <https://falco.org/docs/>, 2026, accessed: 2026-04-09.
- [22] “Prometheus documentation,” <https://prometheus.io/docs/>, 2026, accessed: 2026-04-09.
- [23] “Grafana loki documentation,” <https://grafana.com/docs/loki/latest/>, 2026, accessed: 2026-04-09.
- [24] “Grafana documentation,” <https://grafana.com/docs/grafana/latest/>, 2026, accessed: 2026-04-09.
- [25] “Grafana tempo documentation,” <https://grafana.com/docs/tempo/latest/>, 2026, accessed: 2026-04-09.
- [26] “Opentelemetry documentation,” <https://opentelemetry.io/docs/>, 2026, accessed: 2026-04-09.